

C# Advanced .Net Server-Side Technologies Documentation

Prepare and submit a proposal for the Project with the following structure:

1. Title page

- The name of the project : TVTrack
- Name of the team : Group 8
- The name of each team member :

Austin Nicholson

Jeffrey Pollock

Muhammed Cengiz

Sebastion Da Silva

2. Project Overview

CineTrack (Project) is a web-based movie and TV tracking platform inspired by Letterboxd. The application allows users to search for TV shows using the TMDB API, rate and review shows, add them to their profile, create custom lists, and interact socially by following other users.

The system will be developed using:

- Visual Studio 2022
- ASP.NET Core MVC (.NET 8)
- SQL Server 2022
- Entity Framework Core

- TMDB REST API

The application follows a layered architecture separating presentation logic, business logic, and data access logic. The external TMDB API is used to retrieve show information dynamically.

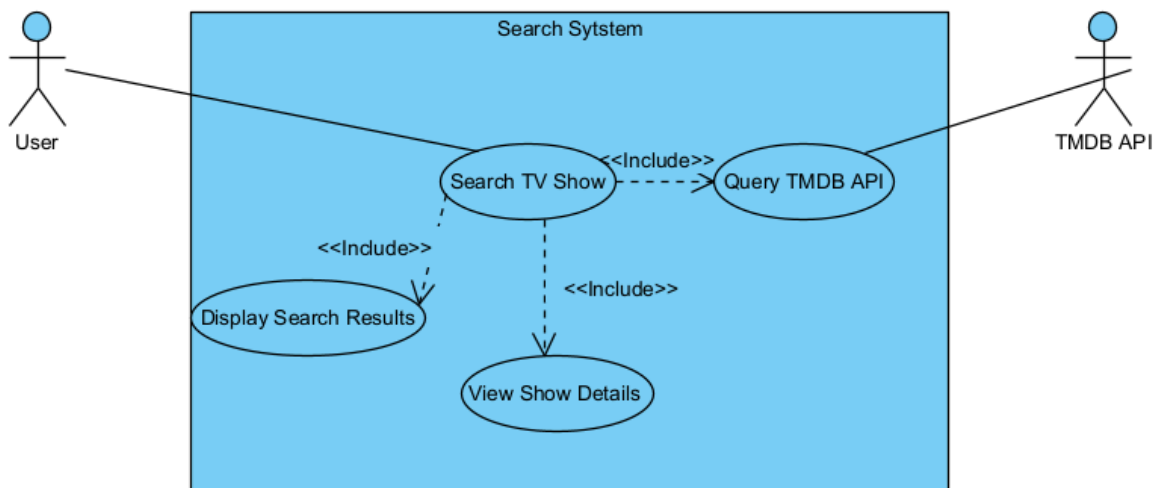
The goal of this project is to demonstrate advanced server-side development concepts including API integration, layered architecture, database design, and collaborative development.

3. Software Model

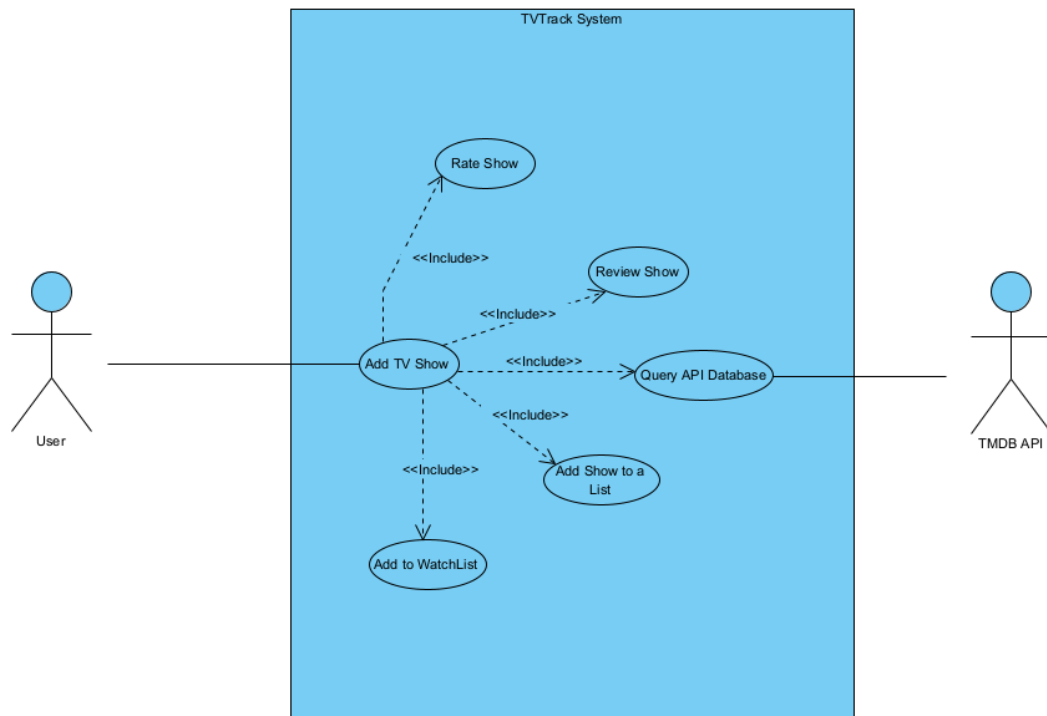
Create and submit a software model built using **Visual Paradigm** with the following details:

a. Use-Case diagrams

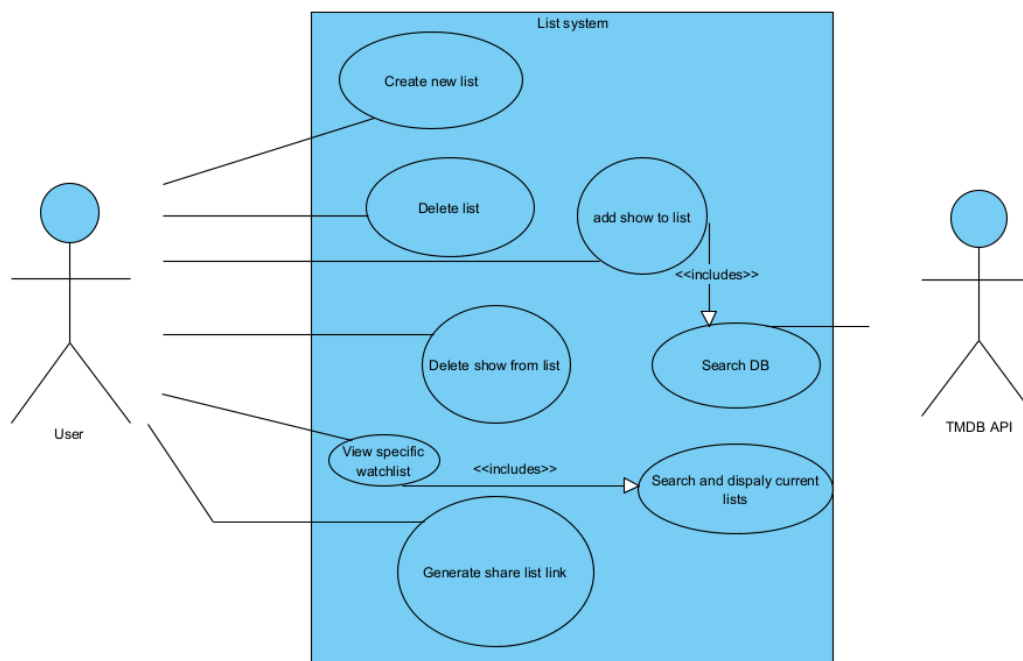
1. Search (Muhammed)



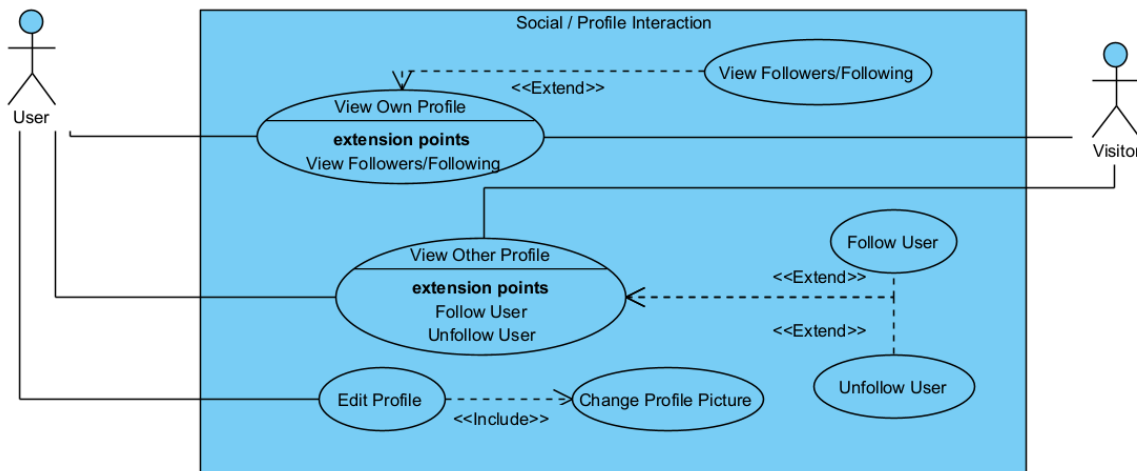
2. Add Show to Account (Jeff)



3. Lists (Sebastian)



4. Social Interaction (Austin)



b. Use-Case Details

For each use-case in the model, use the **Use-Case Details editor** to enter the main steps of the use-case using the **Flow of Events editor**.

Flow of Events for each use case diagram:

1. Search (Muhammed)

Search Tv Show

Search TV Show

Info

Use Case Notes

Flow of Events

Details

Requirements

Diagrams

Test Plan

References

Main Scenario
1.User enters a search keyword.
2.User clicks the search button....

Alternate Flow – No Results Found
1.System receives no matching results from TMDB API. 2.System displays "No..."

+

F

FF

^

v

≡

≡

↶

↷

↺

↻

📄

🔍

🏠

1. User enters a search keyword.

2. User clicks the search button.

3. System validates input.

4. System sends request to TMDB API.

5. System retrieves matching results.

6. System displays results.

7. User selects a show.

Search TV Show alternative scenario

Search TV Show

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Main Scenario
1. User enters a search keyword.
2. User clicks the search button....

Alternate Flow – No Results Fo...
1. System receives no matching results from TMDB API. 2. System displays "No..."

1. System receives no matching results from TMDB API.
2. System displays "No results found" message.
3. User may enter a new search keyword.

Query TMDB API

Query TMDB API

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1.

1. System sends API request with search keyword.
2. TMDB API processes the request.
3. TMDB API returns JSON response.
4. System parses and extracts relevant data.

Display Search Results

Display Search Results

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
 1. System formats retrieved data.
 2. System displays title, poster, and...

1. System formats retrieved data.
 2. System displays title, poster, and overview.
 3. System allows user to select a show.

2. Add Show to Account (Jeff)
 - a. Add Show

Add TV Show

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
 1. User Selects Show from Search
 2. SYSTEM Presents popup with...

1. User Selects Show from Search
 2. **SYSTEM** Presents popup with options
 3. User selects Add Show
 4. **SYSTEM** Adds show to user show database
 5. **SYSTEM** Redirects user to profile page
 6.

- b. Rate Show

Rate Show

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
 1. User selects Rate Show
 2. SYSTEM Presents

1. User selects Rate Show on Add Show popup
 2. **SYSTEM** Presents 5 empty stars
 3. User rates show out of 5 stars
 4. **SYSTEM** Saves rating
 5. **SYSTEM** Redirects user back to main Add Show page

c. Review Show

Review Show

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. User s

1. User selects Review Show from Add Show popup
2. **SYSTEM** Presents text area for user input
3. User enters a review of the show
4. **SYSTEM** Saves review to show
5. **SYSTEM** Redirects user to main Add Show popup

d. Add to List

Add Show to a List

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1.

1. User selects Add to List from main Add Show popup
2. **SYSTEM** Presents existing lists created by user
3. **if** No existing lists
 - 3.1. Prompt user to create a list**end if**
4. User selects list to add Show to
5. **SYSTEM** Adds show to list
6. **SYSTEM** Redirects user to main Add Show popup

e. Add to Watchlist

Add to WatchList

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1.

1. User selects Add to Watch list on Add Show popup
2. **SYSTEM** Adds show to user watch list
3. **SYSTEM** Redirects user to main Add Show popup

3. Lists (Sebastian)
 - a. Create List

Create new list > Create new list Details

Create new list

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario

1.

1. User selects create list button
2. System displays small text box prompting user for a list name
3. User enters name
4. System displays confirmation message that list has been made

- b. Delete list

Delete list > Delete list Details

Delete list

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario

1. User Selects "Delete list" button
2. System displays confirmation...

1. User Selects "Delete list" button
2. System displays confirmation prompt, confirming if the user wants to delete
3. User selects yes, list is deleted
4. System shows confirmation message

Extension:

2.a. User selects no:

1. System gets rid of prompt, list is not deleted.

Enter an extended behavior (without specified parent)

- c. Add show to list

add show to list > add show to list Details

add show to list

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1.

1. User selects option to add show to list
2. System gives a search
3. User enters name of show
4. System queries TMDb to find show based off name and shows possible results
5. User selects desired show
6. System gives submit button
7. User selects submit, show is added to list

Extension:

4.a. Show is not found

1. System displays message saying no show is found

Enter an extended behavior (without specified parent)

d. Delete show from list

Delete show from list

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1.

1. User selects "delete" option for a show in list
2. System prompts confirmation input
3. User selects yes
4. System deletes show from list
5. System displays confirmation message

Extension:

2.a. User selects no

1. System takes prompt away, show is not deleted.

Enter an extended behavior (without specified parent)

e. Search DB

Search DB

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. System creates search query
2. System takes name input from...

1. System creates search query
2. System takes name input from user and adds to search query
3. System finds results
4. System returns results to be parsed and displayed

Extension:

2.a. No result is found

1. System sends an error message back

Enter an extended behavior (without specified parent)

- f. View specific list

View specific watchlist

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. User selects list from

1. User selects view lists option
2. System searches all lists and displays
3. User selects list they want to view
4. System displays contents of the list

Extension:

- 2.a. User searches for list at top of screen
 1. 2.a.a: list is found
 - 1.1. System displays results
 2. 2.a.b: No lists found
 - 2.1. System displays error that no lists were found

Enter an extended behavior (without specified parent)

- g. Search and display current lists

Search and display current lists

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. System creates search query
2. System takes input from user...

1. System creates search query
2. System takes input from user search for name, adds to query
3. System looks for matching list in DB
4. System sends matching list back

Extension:

- 3.a. System cant find list with matching name
 1. System sends back error message to display

Enter an extended behavior (without specified parent)

- h. Generate share list link

=

Generate share list link

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. User selects share link option for list
2. System generates link to list

1. User selects share link option for list
2. System generates link to list
3. System displays link on screen
4. User copies link to their clipboard on device

4. Social Interaction (Austin)
 - a. View Own Profile

View Own Profile

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. User navigates to "My Profile".
2. SYSTEM SYSTEM verifies the us...

1. User navigates to "My Profile".
2. **SYSTEM** SYSTEM verifies the user is authenticated.
3. **SYSTEM** SYSTEM retrieves the user's profile information.
4. **SYSTEM** SYSTEM retrieves the user's stats (following/followers, ratings/saved shows)
5. **SYSTEM** SYSTEM retrieves activity
6. **SYSTEM** SYSTEM displays user's profile page with available options.

i.

- b. View Followers/Following

View Followers/Following

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. User views a profile page (own or other)
2.

1. User views a profile page (own or other)
2. User selects "Followers" or "Following"
3. **SYSTEM** determines which list the user requested (check Followers or Following)
4. **SYSTEM** retrieves corresponding list from the database
5. **SYSTEM** displays the list of users
6. User selects a user from the list
7. **SYSTEM** redirects to profile page of the selected user

i.

- c. View Other Profile

View Other Profile

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario
1. User searches for a user or selects a user from a list
2. SYSTEM loads...

1. User searches for a user or selects a user from a list
2. **SYSTEM** loads selected user's profile
3. **SYSTEM** retrieves the user's profile information
4. **SYSTEM** retrieves stats (followers/following, shows)
5. **SYSTEM** retrieves user's activity
6. **SYSTEM** determines relationship between current user and destination user
7. **SYSTEM** displays profile page with optional action buttons
 - 7.1. "Follow" if not currently following
 - 7.2. "Unfollow" if already following

i.

- d. Follow User

i.

e. Unfollow User

Follow User

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario

1. User views another user's profile
2. User selects "Follow" 3. SYSTEM...

1. User views another user's profile
2. User selects "Follow"
3. **SYSTEM** validates action (checking if the user is logged in, if the target user can be reached)
4. **SYSTEM** creates "follow" relationship record
5. **SYSTEM** updates follower/following counts for each user
6. **SYSTEM** updates page button to ~~unfollow~~ on page refresh

i.

f. Edit Profile

Unfollow User

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario

1. User views another user's profile
2. User selects "Unfollow" 3.

1. User views another user's profile
2. User selects "~~Unfollow~~"
3. **SYSTEM** validates the action (checking if the user is logged in, if the target user is reachable)
4. **SYSTEM** removes "follow" relationship record
5. **SYSTEM** updates the profile page button to "follow" on page refresh

i.

g. Change Profile Picture

Edit Profile

Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario

1. User navigates to "edit profile" from their profile page 2. SYSTEM...

1. User navigates to "edit profile" from their profile page
2. **SYSTEM** verifies the user is signed in
3. **SYSTEM** displays profile edit form populated with all current values
4. User updates a field
5. User selects "save changes"
6. **SYSTEM** validates the input
7. **SYSTEM** saves updated profile information to the user's profile in the database
8. **SYSTEM** redirects the user back to "my profile" with updated information

i.

Change Profile Picture

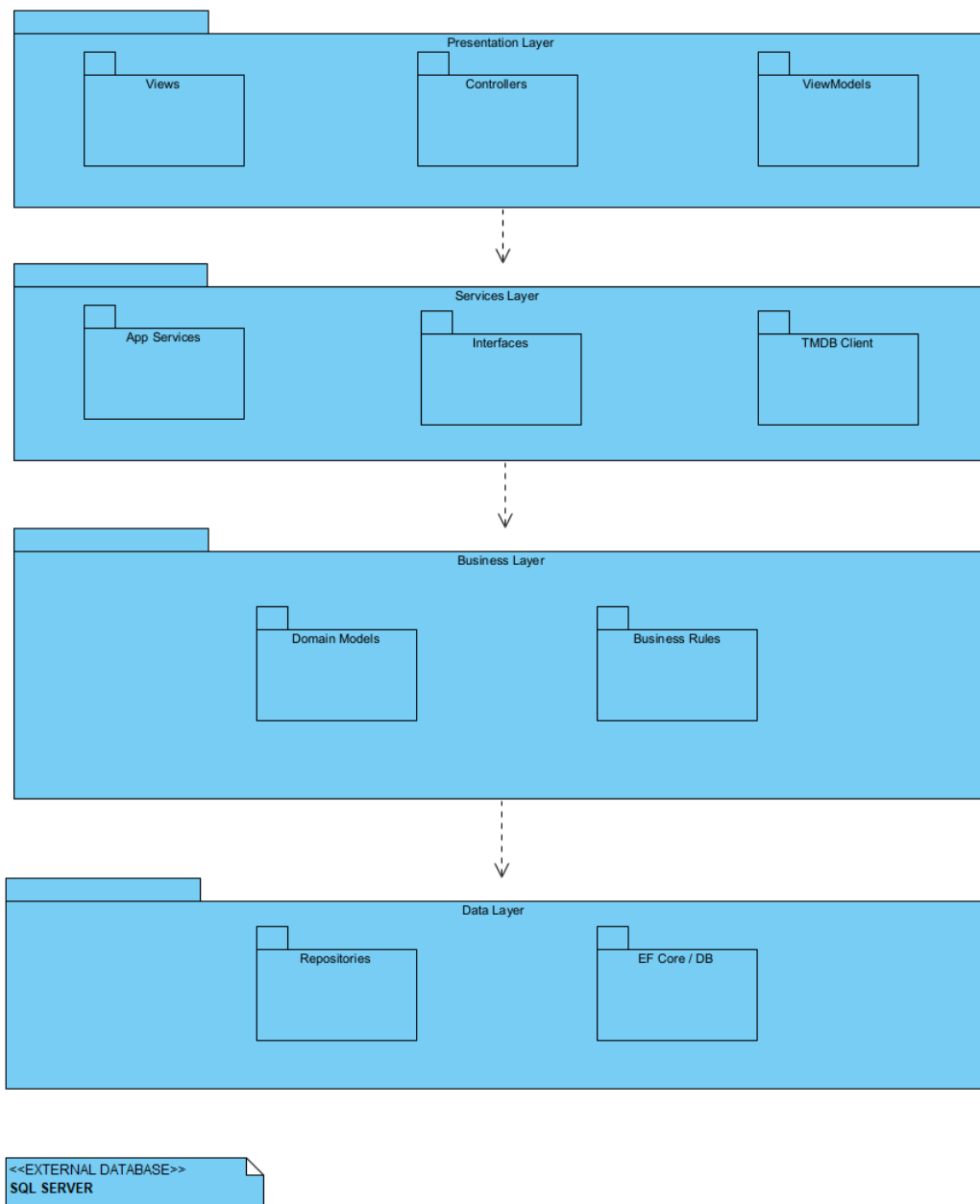
Info Use Case Notes Flow of Events Details Requirements Diagrams Test Plan References

Scenario

1. User navigates to "change profile picture" (from edit profile page)...

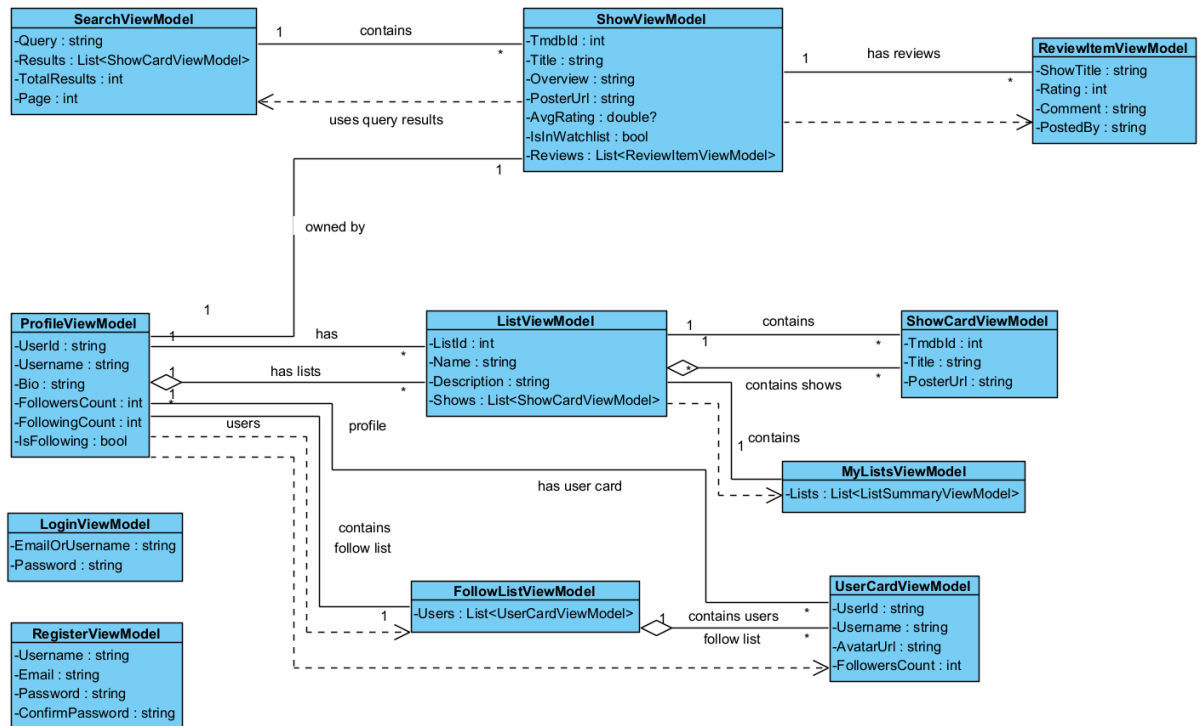
1. User navigates to "change profile picture" (from edit profile page)
2. **SYSTEM** verifies the user is signed in
3. **SYSTEM** displays an upload control for selecting an image
4. User selects the image file and clicks submit
5. **SYSTEM** validates the file format
6. **SYSTEM** stores the image reference in the database
7. **SYSTEM** updates user profile picture, redirects a reloaded page with updated display

c. Package diagram

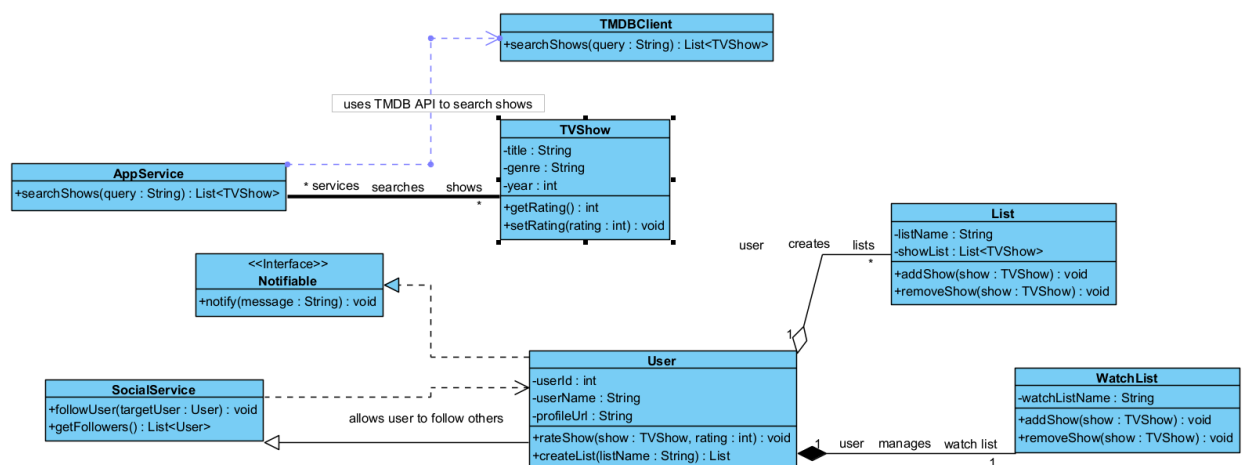


d. Class Diagrams inside each package

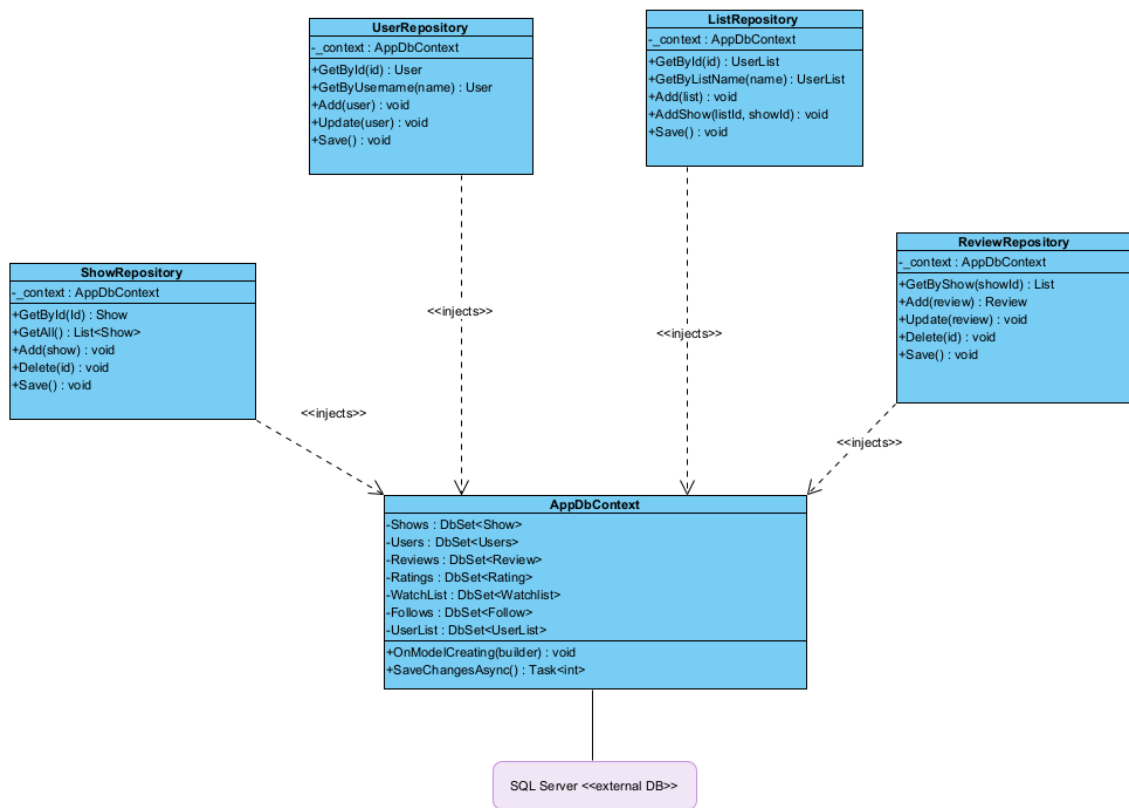
Presentation Layer Class Diagram (Austin)



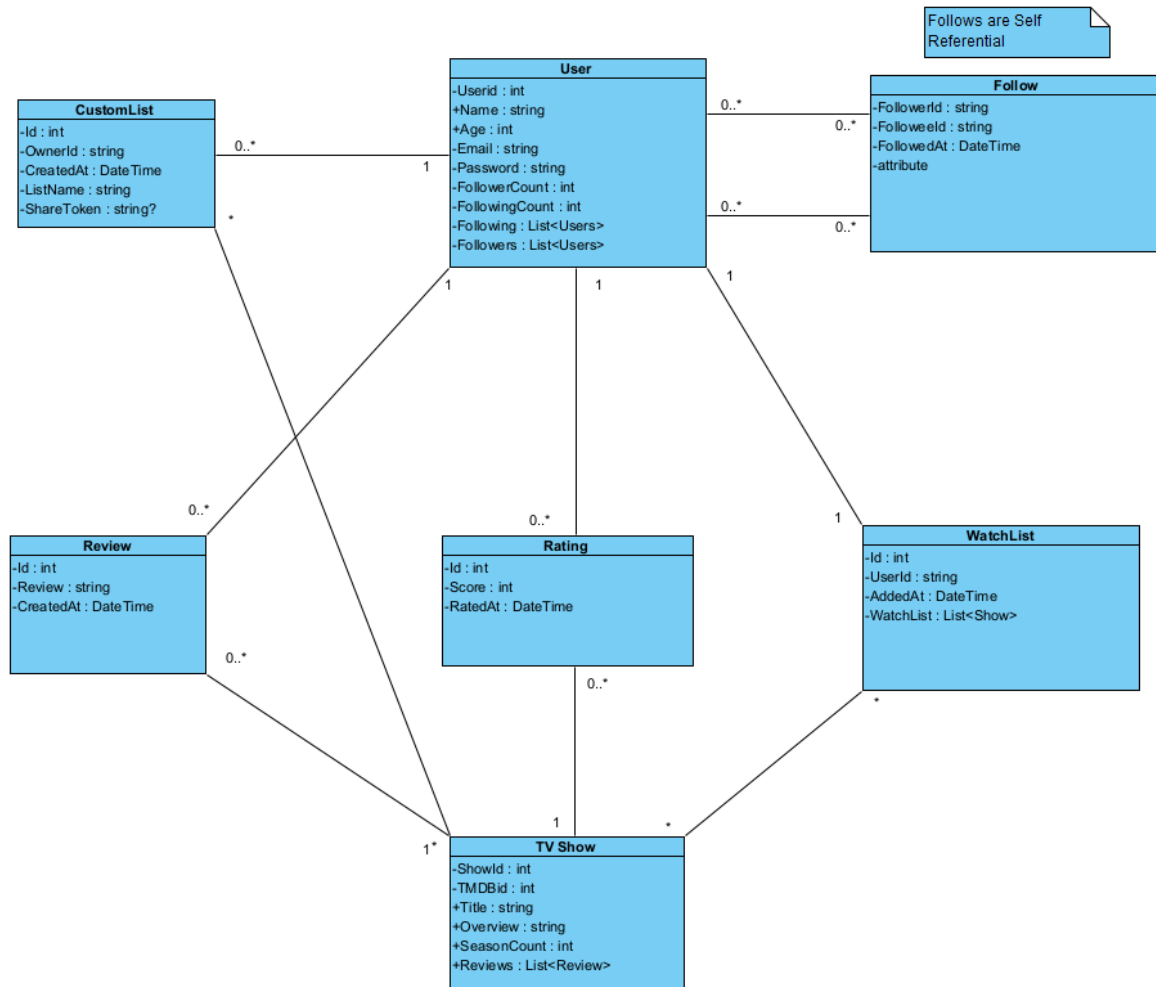
Service Layer Class Diagram (Sebastian)



Data Layer Class Diagram (Jeff)



Business Layer Class Diagram (Muhammed)



These diagrams are very preliminary and should help brainstorm the design of the system.

4. Work Assignments

For each member of the team define:

a. Use-case responsibility

The use-case (functional requirements defined in point 3) each team member is responsible for.

Search ::: Muhammed Cengiz

Add Show to Account ::: Jeffrey Pollock

Lists ::: Sebastion Da Silva

Social Interaction ::: Austin Nicholson

b. Class responsibility

What classes (defined in the class diagram above) each team member is going to design and develop to implement the corresponding use-case.

△ Work assignments must be all-encompassing, with each team member having to implement:

- UI
- Business logic
- Database programming
for the features they are responsible for.

Views ::: Austin Nicholson

Controllers ::: Jeffrey Pollock

Models ::: Muhammed Cengiz

Services ::: Sebastion Da Silva